Database per Service Pattern: Polyglot Persistence & Event Sourcing

Decomposition of Schema Isolation, Polyglot Datastores, Distributed Joins, and CQRS Materialization

EXECUTIVE SUMMARY

The Database per Service pattern establishes true microservice autonomy by ensuring each service exclusively owns its data store. It eliminates cross-domain schema locking, isolates hardware failure domains, and unlocks polyglot persistence. However, it trades away atomic relational joins and immediate ACID consistency for eventual consistency and asynchronous event synchronization.

1. THE FOUR TENETS OF DATABASE ISOLATION

1. Encapsulated Persistence

Database tables are private internal state. Other services can only access data through public APIs (gRPC/REST) or domain events.

2. Polyglot Freedom

Each squad selects the storage engine best suited to its query patterns (Relational, Document, Graph, Time-Series, or Key-Value).

3. Blast Radius Fencing

A corrupted table, slow query, or locked index in Service A cannot exhaust connection pools or crash Service B's database.

4. Independent Schema Evolution

Teams run database migrations, add columns, or alter table schemas in CI/CD pipelines without cross-team release meetings.

2. SOCIAL NETWORK POLYGLOT STORAGE ARCHITECTURE



3. THE DISTRIBUTED JOIN DILEMMA & SOLUTION PATTERNS

Because SQL JOIN across physical database instances is impossible, systems resolve cross-boundary queries using two primary paradigms:

Pattern	Operating Mechanism	Trade-offs
API Composition	API Gateway queries Service A, B, and C concurrently via RPC and merges JSON in memory.	Tail Latency Bounded by slowest service; high network chat.
CQRS Materialized Views	Read service consumes Kafka CDC events and updates a denormalized view in Elasticsearch/Mongo.	Optimal Read Ultra-fast queries; eventual consistency lag.

4. CHANGE DATA CAPTURE (CDC) ARCHITECTURE WITH DEBEZIUM

CDC eliminates application dual-writes by tailing native transaction logs:

```
// Debezium Kafka Connect Configuration for PostgreSQL
{
  "name": "orders-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "tasks.max": "1",
    "database.hostname": "orders-db.internal",
    "database.port": "5432",
    "database.user": "debezium",
    "database.dbname": "orders_db",
    "database.server.name": "orders_cdc",
    "table.include.list": "public.orders,public.order_items",
    "plugin.name": "pgoutput"
  }
}
```

5. EVENT SOURCING VS. STATE-BASED PERSISTENCE

Traditional State-Based Storage

Overwrites rows via `UPDATE balance = 50`. Previous states are permanently lost. Audit trails require separate custom logging tables.

Event Sourcing (Log of Record)

Appends immutable events: `AccountOpened` → `MoneyDeposited($100)` → `MoneyWithdrawn($50)`. Replay guarantees 100% mathematical auditability.

6. OPERATIONAL REALITIES OF MANAGING 50+ DATASTORES

Backup & Recovery

Restoring a microservice ecosystem to a specific timestamp requires synchronized point-in-time recovery (PITR) across dozens of instances.

Connection Pool Sizing

Multiple service replicas scaling out in Kubernetes can overwhelm database connection limits. Requires PgBouncer or AWS RDS Proxy.

Schema Drift Monitoring

Automated CI linters (Flyway / Liquibase) must enforce non-breaking additive schema changes (never drop columns without deprecation cycles).

7. DECLARATIVE MULTI-DATABASE TERRAFORM INFRASTRUCTURE

```
# AWS Terraform: Isolated RDS Instance per Service
module "orders_db" {
  source         = "terraform-aws-modules/rds/aws"
  identifier     = "orders-service-db"
  engine         = "postgres"
  engine_version = "15.4"
  instance_class = "db.t4g.medium"
  allocated_storage = 50
  vpc_security_group_ids = [aws_security_group.orders_db_sg.id]
  # Fenced: Subnet only accessible by orders-service pods
  subnet_ids     = module.vpc.database_subnets
}
```

8. PRODUCTION DATABASE ISOLATION CHECKLIST

Database per Service Standards

1. **No Shared Credentials:** No service may possess database credentials to any other service's datastore.
2. **CDC over Dual-Writes:** Use Debezium or transactional outbox for cross-service data replication.
3. **Connection Pooling:** Deploy connection poolers (PgBouncer/ProxySQL) in front of relational databases.
4. **Disaster Recovery Drills:** Regularly simulate multi-service database restoration to verify eventual consistency convergence.